The Microservice-ization of gem5

Pomona College

Connor Wang, Samuel Thomas

gem5 Execution Model

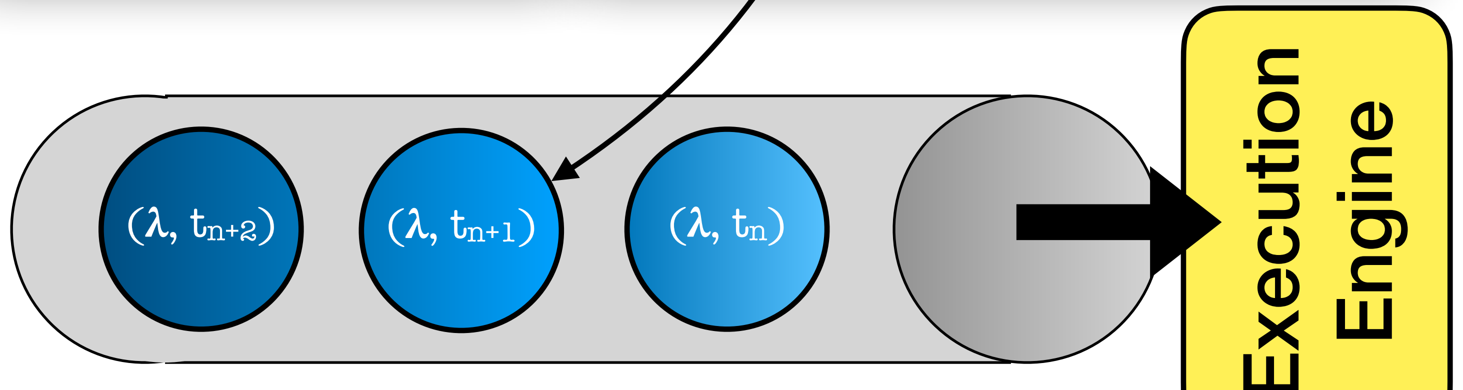
```
from m5 import *

# ...

system.memory.mem_ctrl.dram = \
   DDR4_2400_8x8()
system.memory.mem_ctrl.port = \
   system.membus.mem_side_ports

# ...
```

Front End

Back End

```
class MemCtrl
{
  private:
    // ...

  public:
    MemCtrl() {};

    // ...
};
```

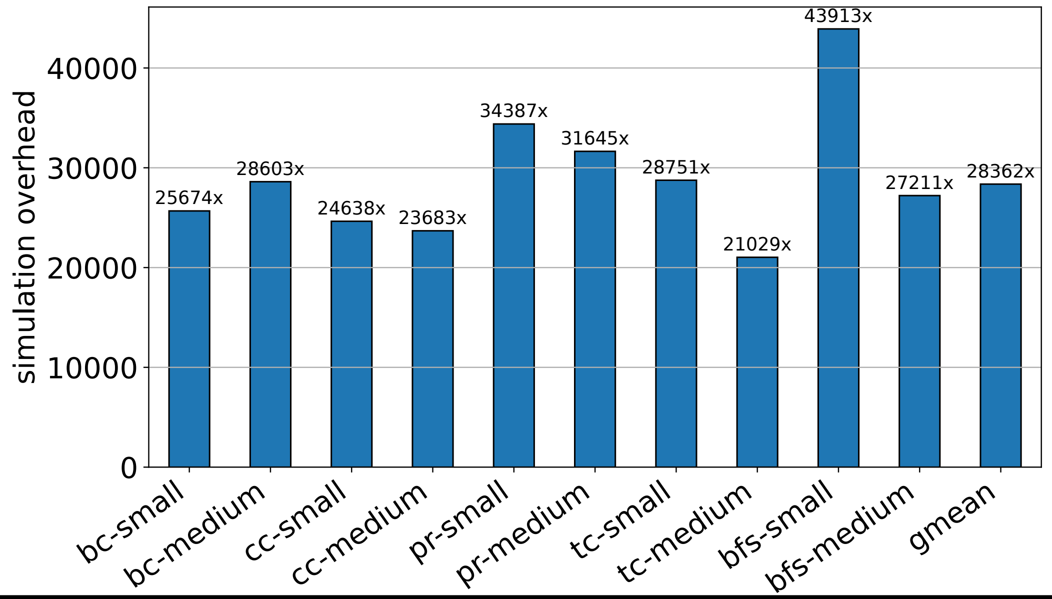
```
// ...

void
MemCtrl::recvReq()
{
   schedule(
         dram.accessEvent,
         curTick() + latency
   );
}
```



Event Queue

Motivation

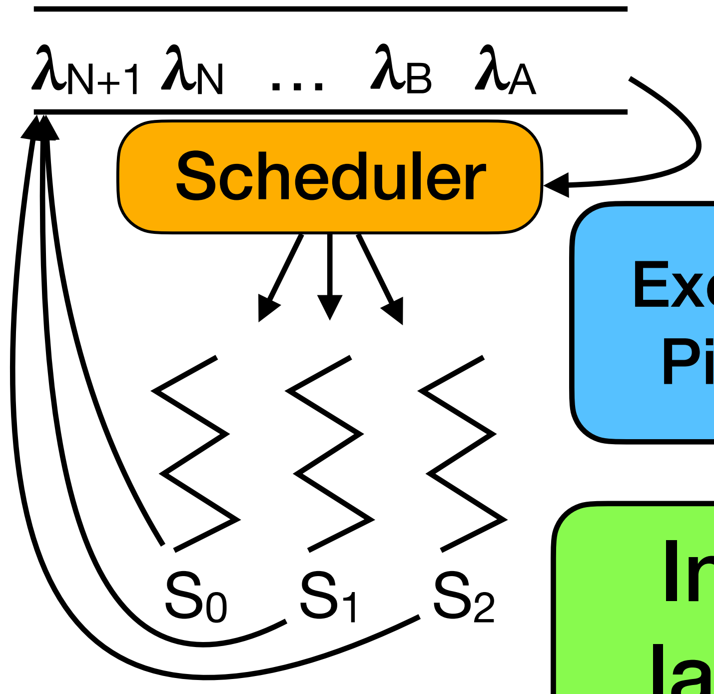
Way too slow!



Takes hours to simulate seconds with several GB of memory!

Typical evaluations perform hundreds of simulations!

Proposal



Execution Pipeline

Scheduling Challenges

Work partitioning strategy

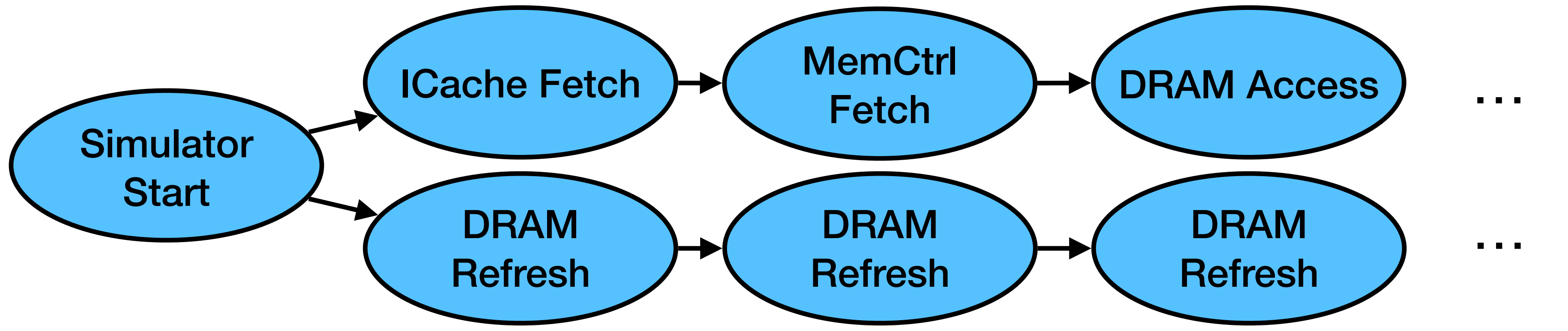
Synchronization of simulated time

Management of dependent events

Intuition: execute lambda events on “service” threads

Insight

Sample Event Call Trace



Independent branches of execution based on hardware behavior

Component-wise request level parallelism

Expected Outcomes

Reduced on-host execution time!

Improved “event throughput”

Consistent resource utilization

Benefit thousands of users

Open Questions

Request-level or component-level parallelism?

Scalability of worker threads?

Abstraction towards different workloads and configurations?

Expected Challenges

Loss of execution determinism

Increased debugging complexity